AMR Extension of LaRT: Octree Structure and Raytrace Algorithm

Contents

1. Overview **2**

2. AMR Grid Data Structure **2**

2.1 Octree Topology . . . 2

2.2 Arrays Stored per Cell . . . 3

2.3 Coordinate Convention . . . 3

3. Building the Octree **3**

3.1 Tree Construction: `amr_build_tree` . . . 3

3.2 Face-Neighbour Precomputation: `amr_build_neighbors` . . . 4

4. Cell-Crossing Algorithms **4**

4.1 Finding a Leaf: `amr_find_leaf` . . . 4

4.2 Cell-Exit Distance: `amr_cell_exit` . . . 5

4.3 Next Leaf After a Face Crossing: `amr_next_leaf` . . . 5

5. Grid Initialisation **5**

5.1 Input Data Formats . . . 6

5.2 Tau Normalisation by Pole Traversal . . . 7

6. Photon Propagation **7**

6.1 Main Raytrace Routine: `raytrace_to_tau_amr` . . . 7

6.2 Frequency Shift at Cell Boundaries . . . 8

6.3 Overshoot Correction . . . 8

6.4 Photon Escape and Spectrum Accumulation . . . 9

6.5 Other Raytrace Routines . . . 9

7. Scattering **9**

7.1 Resonance Scattering . . . 9

7.2 Dust Scattering . . . 10

7.3 Multi-Level Resonance Transitions . . . 10

8. Output Normalisation **10**

9. Core-Skip Acceleration **11**

10.Validation **11**

11.Summary of Key Design Choices **12**

1. Overview

LaRT (Lyman-alpha Radiative Transfer) is a Monte Carlo radiative transfer code for Lyman-α photon propagation in astrophysical media. The base code (`LaRT_combined`) uses a uniform Cartesian grid. `LaRT_v2.00` extends it with an Adaptive Mesh Refinement (AMR) octree grid, enabling direct use of outputs from cosmological simulation codes such as RAMSES.

AMR mode is activated with `par%use_amr_grid = .true.` in the input namelist. All photon-physics modules (scattering, raytrace, peel-off, output) are selected through Fortran procedure pointers set in `setup_v2c.f90`; the outer simulation loop (`run_simulation_mod_v2a`) and photon injection (`gen_photon_car_v2b`) are therefore unchanged.

The new source files are:

File	Role
`octree_mod.f90`	Octree data type, tree operations, cell-crossing routines
`grid_mod_amr.f90`	Grid creation, opacity normalisation, frequency grid
`raytrace_amr.f90`	Photon propagation through AMR cells
`scattering_amr.f90`	Resonance and dust scattering (per-cell parameters)
`peelingoff_amr.f90`	Peel-off observer calculation
`read_ramses_amr.f90`	Reader for RAMSES binary and generic text AMR formats

2. AMR Grid Data Structure

2.1 Octree Topology

The AMR grid is represented as a *linear octree*: a flat Fortran array of cells indexed $1, 2, \ldots, N_{\text{cells}}$. Cell 1 is the root, which covers the entire computational box. Each internal cell has exactly eight children obtained by bisecting the parent along all three coordinate axes.

The child octant index is encoded as

$$i_{\text{oct}} = 1 + i_x + 2\,i_y + 4\,i_z, \tag{1}$$

where $i_x = 1$ if $x \geq c_x$ (the cell centre), otherwise $i_x = 0$, and similarly for i_y and i_z. Hence octant indices run from 1 to 8.

2.2 Arrays Stored per Cell

The Fortran derived type `amr_grid_type` (defined in `octree_mod.f90`) stores the following arrays of size N_{cells} for the tree topology:

Array	Dimension	Content
`parent(:)`	N_{cells}	parent cell index (0 for root)
`children(:,:)`	$8 \times N_{\text{cells}}$	child indices; 0 = leaf
`level(:)`	N_{cells}	refinement level (0 = root)
`cx, cy, cz`	N_{cells}	cell centre coordinates (code units)
`ch(:)`	N_{cells}	cell half-width (code units)
`ileaf(:)`	N_{cells}	> 0: leaf index; 0: internal
`icell_of_leaf(:)`	N_{leaf}	cell index for each leaf
`neighbor(:,:)`	$6 \times N_{\text{cells}}$	face-neighbour table (see Section 3.2)

Leaf cells (`ileaf`$(i) > 0$) carry physical data stored in separate arrays of size N_{leaf}:

Array	Symbol	Physical meaning
`rhokap`	$\bar{\kappa}_0$	HI line-centre opacity per code-unit length
`rhokapD`	$\bar{\kappa}_d$	dust opacity per code-unit length
`Dfreq`	$\Delta\nu_D$	local Doppler frequency [Hz]
`voigt_a`	a	Voigt damping parameter $a = A_{21}/(4\pi\Delta\nu_D)$
`vfx, vfy, vfz`	$\tilde{v}_x, \tilde{v}_y, \tilde{v}_z$	bulk velocity / local v_{th}

The spectral output arrays (`Jout`, `Jin`, `Jabs`) have size N_ν (the number of frequency bins) and are also members of `amr_grid_type`.

2.3 Coordinate Convention

All cell positions and path lengths are stored in *code units* (kpc, pc, or whatever unit the input data use). The conversion factor `par%distance2cm` is used once when computing opacities:

$$\bar{\kappa}_0\ [\text{code-unit}^{-1}] = n_{\text{HI}}\,[\text{cm}^{-3}] \times \frac{\sigma_0}{\Delta\nu_D} \times d_{\text{cm}}, \tag{2}$$

where d_{cm} is the number of centimetres per code unit. After that, all quantities are dimensionless with respect to code units, exactly mirroring the Cartesian convention.

3. Building the Octree

3.1 Tree Construction: `amr_build_tree`

`amr_build_tree` takes flat arrays of leaf-cell centre coordinates (`xleaf`, `yleaf`, `zleaf`), AMR levels (`leaf_level`), and box bounds, and constructs the octree by inserting each leaf cell top-down from the

root.

The algorithm is as follows:

1. Initialise the root cell (index 1) with half-width $h = L_{\text{box}}/2$.
2. For each leaf $\ell = 1, \ldots, N_{\text{leaf}}$:
   (a) Start at the root (`icell` $= 1$).
   (b) For each refinement level from 0 to $\ell_{\text{leaf}} - 1$:
      i. Compute the octant index i_{oct} using the leaf centre.
      ii. If `children(`i_{oct}`, icell)` is zero, create a new internal node with half-width $h/2$ and centre offset $\pm h/2$ from the parent centre.
      iii. Descend: `icell` $\leftarrow$ `children`$(i_{\text{oct}}, \texttt{icell})$.
   (c) Mark the reached cell as leaf ℓ: `ileaf(icell)` $= \ell$.

The array is grown dynamically (doubled) if the pre-allocated size is exceeded.

3.2 Face-Neighbour Precomputation: `amr_build_neighbors`

After the tree is built, `amr_build_neighbors` pre-computes a $6 \times N_{\text{cells}}$ neighbour table. Entry `neighbor`(f, i) stores the cell index of the neighbour of cell i across face f, using the face convention:

$$f = 1\colon +x, \quad 2\colon -x, \quad 3\colon +y, \quad 4\colon -y, \quad 5\colon +z, \quad 6\colon -z. \tag{3}$$

A value of 0 means the face is on the domain boundary.

For each cell i, the routine probes the point just outside each face (offset $h \times 10^{-8}$ from the face) and descends the tree to the cell at level $\leq \ell_i$ that contains that probe point:

$$\texttt{neighbor}(f, i) = \texttt{amr_find_cell_at_level}(c_i + h_f \hat{e}_f,\ \ell_i)\,, \tag{4}$$

where h_f is the signed probe offset along face direction f and $\hat{e}_f$ is the corresponding unit vector.

The result may be a leaf cell at the same level, a leaf at a coarser level (if the grid is not refined there), or an internal cell at the same level (if the neighbour side is *more* refined than cell i). The last case is handled at runtime by `amr_next_leaf` (Section 4.3).

This table is built once and then used for O(1) boundary crossings throughout the simulation.

4. Cell-Crossing Algorithms

4.1 Finding a Leaf: `amr_find_leaf`

`amr_find_leaf`(x, y, z) descends the octree from the root, choosing the correct child octant at each level using the octant formula (Equation 1), until it reaches a cell with `ileaf` > 0. The result is the leaf index $\ell \in \{1, \ldots, N_{\text{leaf}}\}$. The depth is at most $\ell_{\max}$, the maximum AMR level.

This function is used only at photon injection (once per photon) and as a fallback if `photon%icell_amr` is not yet set.

4.2 Cell-Exit Distance: `amr_cell_exit`

Given a photon at position (x, y, z) inside cell i with direction (k_x, k_y, k_z), `amr_cell_exit` computes the distance t_{exit} to the nearest cell face and identifies which face f is crossed.
For each axis $\alpha \in \{x, y, z\}$ with cell centre c_α and half-width h, the distances to the $\pm$ faces are:

$$t_\alpha^+ = \frac{c_\alpha + h - \alpha}{k_\alpha}, \quad t_\alpha^- = \frac{c_\alpha - h - \alpha}{k_\alpha}, \tag{5}$$

with $t = \infty$ (implemented as `hugest`) when the direction component is zero or points away from the face. The exit is the minimum over all six half-space times:

$$t_{\text{exit}} = \min(t_x^+,\, t_x^-,\, t_y^+,\, t_y^-,\, t_z^+,\, t_z^-), \tag{6}$$

and f is the index of the minimising element (1–6 in the convention above).

4.3 Next Leaf After a Face Crossing: `amr_next_leaf`

After the photon crosses face f of cell i and the position has been updated to (x, y, z) at the face, `amr_next_leaf`(i, f, x, y, z) returns the leaf index of the cell the photon enters. The algorithm is:

1. Look up `ineigh` = `neighbor`(f, i). If zero, the photon has left the domain; return 0.
2. If `ileaf(ineigh)` > 0, the neighbour is already a leaf; return `ileaf(ineigh)`.
3. Otherwise the neighbour is an internal cell at a finer level. Descend:
   (a) Compute i_{oct} from the actual crossing position (x, y, z) and the centre of `ineigh`.
   (b) `ineigh` $\leftarrow$ `children`$(i_{\text{oct}}, \texttt{ineigh})$.
   (c) Repeat until `ileaf(ineigh)` > 0.
4. Return `ileaf(ineigh)`.

Step 1 is O(1); the descent in step 3 traverses at most $\Delta\ell = \ell_{\max} - \ell_i$ extra levels, which is typically one or two. In practice this makes cell-boundary crossings effectively O(1) for any AMR grid.

5. Grid Initialisation

`grid_create_amr` in `grid_mod_amr.f90` performs the following sequence:

1. **Read leaf-cell data.** MPI rank 0 reads the data file (RAMSES binary or generic text) and broadcasts all arrays to the other ranks. Each rank therefore holds a full copy of the tree (appropriate for moderate grid sizes).

2. **Build octree.** Call `amr_build_tree` and `amr_build_neighbors`.

3. **Compute per-cell Doppler and Voigt parameters.** For leaf ℓ:

$$\Delta\nu_{D,\ell} = \frac{v_{\mathrm{th},1}\sqrt{T_\ell}}{\lambda_0}, \quad a_\ell = \frac{A_{21}}{4\pi\,\Delta\nu_{D,\ell}}, \tag{7}$$

   where $v_{\mathrm{th},1} = \sqrt{2k_B/m_H}$ is the thermal velocity per $\sqrt{T}$.

4. **Compute per-cell opacity and velocity.**

$$\bar{\kappa}_{0,\ell} = n_{\mathrm{HI},\ell}\,\frac{\sigma_0}{\Delta\nu_{D,\ell}}\,d_{\mathrm{cm}}, \tag{8}$$

$$\tilde{v}_{\alpha,\ell} = \frac{v_{\alpha,\ell}[\mathrm{km\,s^{-1}}]}{v_{\mathrm{th},\ell}[\mathrm{km\,s^{-1}}]}, \tag{9}$$

   where σ_0 is the hydrogen Lyman-α cross-section at line centre and d_{cm} is centimetres per code unit.

5. **Normalise opacity to `taumax`.** See Section 5.2.

6. **Set up frequency grid and output arrays.**

5.1 Input Data Formats

Two readers are provided.

RAMSES format (`amr_type = 'ramses'`). Reads the standard RAMSES AMR and hydro binary output files from `output_XXXXX/`. Unit scales (`unit_l`, `unit_d`, `unit_t`) are taken from `info_XXXXX.txt`. The routine returns leaf positions in cm and n_H in cm^{-3}.

Generic text format (`amr_type = 'generic'`). A plain-text file with a header followed by one line per leaf cell:

```
# comment lines start with #
BOXLEN  <box_length_in_code_units>
NLEAF   <number_of_leaf_cells>
# x  y  z  level  nH[cm^-3]  T[K]  vx[km/s]  vy[km/s]  vz[km/s]
<x>  <y>  <z>  <level>  <nH>  <T>  <vx>  <vy>  <vz>
...
```

The code unit is set by `par%distance_unit` (e.g., `'kpc'`).

5.2 Tau Normalisation by Pole Traversal

The optical depth normalisation convention in `LaRT_v2.00` matches the Cartesian version: `taumax` is the line-centre optical depth from the box centre to the $+z$ boundary along the z-axis.
After the raw opacities are set, a ray is traced from the box centre in the $+z$ direction using the same AMR raytrace kernel:

$$\tau_{\rm pole} = \sum_{\ell} \bar{\kappa}_{0,\ell}\, \phi(0, a_\ell)\, \Delta s_\ell, \tag{10}$$

where $\phi(x, a)$ is the Voigt profile at $x = 0$ and Δs_ℓ is the path-length through leaf ℓ along the ray. The normalisation factor is

$$f_{\rm norm} = \frac{\texttt{taumax}}{\tau_{\rm pole}}, \tag{11}$$

and all opacities are scaled: $\bar{\kappa}_{0,\ell} \leftarrow f_{\rm norm}\, \bar{\kappa}_{0,\ell}$. The same factor is applied to dust opacity if present.
This pole-traversal approach is exact for any opacity distribution, in contrast to the volume-average approximation $\tau \approx \langle \bar{\kappa}_0 \rangle\, L_{\rm box}$ that underestimates $\tau_{\rm pole}$ for centrally concentrated clouds such as a sphere.

6. Photon Propagation

6.1 Main Raytrace Routine: `raytrace_to_tau_amr`

This routine propagates a photon through the AMR grid until it accumulates a target optical depth $\tau_{\rm in}$ (sampled as $-\ln \xi$ for a uniform random ξ) or leaves the box.
The photon state is represented by:

- (x, y, z) — current absolute position in code units,
- (k_x, k_y, k_z) — unit direction vector,
- x_ν — frequency offset in units of the local Doppler width (comoving frame of the current cell),
- ℓ — current leaf index (`photon%icell_amr`).

The loop at each step n:

1. Convert leaf index to cell index: $i =$ `icell_of_leaf`(ℓ).
2. Compute the exit distance and exit face: $(t_{\rm exit}, f) =$ `amr_cell_exit`$(i, x, y, z, k_x, k_y, k_z)$.
3. Evaluate the opacity at the current frequency:

$$\kappa = \bar{\kappa}_{0,\ell}\, \phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell} \quad \text{(with dust, if present)}, \tag{12}$$

   where $\phi(x, a)$ is the Voigt function.

4. Update accumulated optical depth: $\tau \leftarrow \tau + \kappa\, t_{\rm exit}$.
5. **If** $\tau \geq \tau_{\rm in}$: backtrack to the exact interaction point (Section 6.3) and exit the loop.
6. Otherwise: advance position to the face,

$$(x, y, z) \leftarrow (x + t_{\rm exit}\, k_x,\ y + t_{\rm exit}\, k_y,\ z + t_{\rm exit}\, k_z). \tag{13}$$

7. Look up the next leaf: $\ell' = \texttt{amr_next_leaf}(i, f, x, y, z)$.
8. If $\ell' = 0$, the photon has escaped; set `photon%inside = .false.` and exit.
9. Apply the frequency shift (Section 6.2): $x_\nu \leftarrow x'_\nu$.
10. Set $\ell \leftarrow \ell'$ and continue.

6.2 Frequency Shift at Cell Boundaries

Each cell has its own Doppler frequency $\Delta\nu_{D,\ell}$ (because cells can have different temperatures) and its own bulk velocity component along the photon direction,

$$u_\ell = \tilde{v}_{x,\ell}\, k_x + \tilde{v}_{y,\ell}\, k_y + \tilde{v}_{z,\ell}\, k_z \quad \text{[dimensionless, in units of local } v_{\rm th}\text{]}. \tag{14}$$

When crossing from cell ℓ (old) to cell ℓ' (new), the photon frequency must be transformed to the comoving frame of the new cell. The physical frequency is invariant, so:

$$\nu_{\rm phys} = \nu_0 \left(1 + \frac{(x_\nu^{\rm old} + u_\ell)\, \Delta\nu_{D,\ell}}{\nu_0}\right), \tag{15}$$

and the new dimensionless frequency is:

$$\boxed{x_\nu^{\rm new} = (x_\nu^{\rm old} + u_\ell)\, \frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\ell'}} - u_{\ell'}.} \tag{16}$$

This formula transforms between the comoving frames of two cells with different temperatures and bulk velocities in a single step.

6.3 Overshoot Correction

When $\tau > \tau_{\rm in}$ after stepping across a cell, the photon actually stopped inside the cell. The exact interaction distance is found by backtracking:

$$\Delta s_{\rm back} = \frac{\tau - \tau_{\rm in}}{\kappa}. \tag{17}$$

The total displacement from the entry point is then $d = d + t_{\rm exit} - \Delta s_{\rm back}$, and the final position is

$$(x, y, z) \leftarrow (\texttt{photon\%}x + d\, k_x,\ \texttt{photon\%}y + d\, k_y,\ \texttt{photon\%}z + d\, k_z). \tag{18}$$

Note that the position update is accumulated as a single floating-point addition from the *original* entry point (not from successive face crossings), which preserves numerical precision.

6.4 Photon Escape and Spectrum Accumulation

When the photon leaves the box ($\ell' = 0$), the frequency is converted to the lab frame:

$$x_\nu^{\rm lab} = x_\nu + u_\ell, \tag{19}$$

then scaled to the reference Doppler frequency:

$$x_\nu^{\rm ref} = x_\nu^{\rm lab}\,\frac{\Delta\nu_{D,\ell}}{\Delta\nu_{D,\rm ref}}. \tag{20}$$

The frequency bin index is

$$i_\nu = \left\lfloor \frac{x_\nu^{\rm ref} - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{21}$$

and the photon weight is accumulated into `amr_grid%Jout`(i_ν).

6.5 Other Raytrace Routines

Six additional routines in `raytrace_amr_mod` share the same AMR traversal kernel but accumulate different quantities:

Routine	Purpose
`raytrace_to_edge_amr`	Total τ to box edge (photon unchanged)
`raytrace_to_edge_tau_gas_amr`	Gas-only τ to box edge
`raytrace_to_edge_column_amr`	$N_{\rm HI}$ and dust τ to box edge
`raytrace_to_dist_amr`	Total τ to fixed distance s
`raytrace_to_dist_tau_gas_amr`	Gas-only τ to fixed distance s
`raytrace_to_dist_column_amr`	$N_{\rm HI}$ and dust τ to fixed distance s

All six routines propagate a copy of the photon state (photon is not modified) and apply the same frequency-shift formula at each cell crossing so that the Voigt opacity is evaluated at the correct comoving frequency in every cell.

7. Scattering

The scattering routines in `scattering_amr.f90` mirror the Cartesian versions but read physical parameters from `amr_grid` arrays indexed by the leaf index `photon%icell_amr`.

7.1 Resonance Scattering

`scatter_resonance_nostokes_amr` implements the standard Lyman-α resonance scattering algorithm (no Stokes polarisation):

1. Sample the thermal velocity component of the scattering atom along the photon direction:

$$u_z = x_\nu + u_\ell \quad \text{(in the cell frame)}. \tag{22}$$

   The atom thermal velocity perpendicular to the photon direction is sampled from a Gaussian.

2. Draw a new photon direction isotropically in the atom frame.

3. Compute the new photon frequency in the atom frame, then transform back to the cell comoving frame.

4. Apply the core-skip acceleration if $|x_\nu| < x_{\rm crit}$.

The core-skip threshold $x_{\rm crit}$ is computed in `grid_create_amr` from $a\tau_0^{\rm cell}$, the product of the mean Voigt damping parameter and the mean optical depth per AMR leaf cell.

7.2 Dust Scattering

`scatter_dust_nostokes_amr` uses the standard Henyey–Greenstein phase function. When dust is present, the probability of the interaction being a dust event is

$$p_d = \frac{\bar{\kappa}_{d,\ell}}{\bar{\kappa}_{0,\ell}\,\phi(x_\nu, a_\ell) + \bar{\kappa}_{d,\ell}}, \tag{23}$$

sampled at the scattering location (cell ℓ).

7.3 Multi-Level Resonance Transitions

For complex atoms (HeI, FeII) represented by multiple transitions, the AMR scattering module provides `do_resonance4_amr`, `do_resonance5_amr`, and `do_resonance6_amr`, which follow the same logic as their Cartesian counterparts but use per-cell Doppler frequencies and Voigt parameters.

8. Output Normalisation

After all photons are traced, the MPI reduction collects all per-frequency contributions via `MPI_ALLREDUCE` into `amr_grid%Jout` (`output_reduce_amr` in `output_sum_rect.f90`).
The normalised specific intensity (per unit frequency per steradian) is:

$$J_{\rm out}(x_\nu) = \frac{\texttt{Jout}(i_\nu)}{N_{\rm ph}\;\delta x\;2\pi\;A}, \tag{24}$$

where $N_{\rm ph}$ is the number of photons, δx the frequency bin width, and

$$A = 4\pi\, d_{\rm cm}^2 \tag{25}$$

is the surface area of a unit sphere in cm^2 (the factor $d_{\rm cm}$ is `par%distance2cm`).

For dimensionless runs (`distance2cm = 1`, no `distance_unit`), this reduces to $A = 4\pi$, making $J_{\rm out}$ independent of the arbitrary box size. This matches the Cartesian convention ($A = 4\pi\, r_{\rm max}^2\, d_{\rm cm}^2$) when $r_{\rm max} = L_{\rm box}/2$ and the same `distance2cm` is used.

9. Core-Skip Acceleration

Lyman-α photons scatter extremely frequently in the line core ($|x_\nu| \ll 1$), where the optical depth per cell is large. The core-skip algorithm (Laursen et al. 2009) exploits the fact that a photon in the core will scatter many times within a single cell without travelling far. When $|x_\nu| < x_{\rm crit}$, the frequency is redrawn from the wing distribution directly, skipping the core-scattering steps.
In AMR mode, $x_{\rm crit}$ is computed from the mean opacity product:

$$(a\tau_0)_{\rm cell} = \bar{a}\ \bar{\tau}_0\ /\ N_{\rm cell,axis}, \tag{26}$$

where $N_{\rm cell,axis} = L_{\rm box}/(2\, h_{\rm root})$ is the number of root cells along one axis. The threshold is then

$$x_{\rm crit} = \begin{cases} 0.02\ \exp[\xi\,(\ln(a\tau_0)_{\rm cell})^\chi] & (a\tau_0)_{\rm cell} > 1, \\ 0 & \text{otherwise}, \end{cases} \tag{27}$$

with $(\xi, \chi) = (0.6, 1.2)$ for $(a\tau_0)_{\rm cell} \le 60$ and $(1.4, 0.6)$ otherwise.

10. Validation

The AMR implementation was validated against equivalent Cartesian runs for a uniform-density sphere of HI gas with a central point source ($T = 10^4\,$K, no dust).
The AMR sphere is described by the generic text format with the same set of leaf cells at the same AMR level, providing effectively the same resolution as the 101^3 Cartesian grid. The Cartesian comparison uses $r_{\rm max} = L_{\rm box}/2$ with no `distance_unit`.

τ_0	$J_{\rm out,max}^{\rm AMR}$	$J_{\rm out,max}^{\rm CAR}$	Ratio (AMR/CAR)	Peak velocity
10^2	6.630×10^{-3}	6.836×10^{-3}	0.970	$\pm 26.75\,{\rm km\,s^{-1}}$
10^4	7.144×10^{-3}	7.402×10^{-3}	0.965	$\pm 38.21\,{\rm km\,s^{-1}}$

Peak positions, column densities $N({\rm HI})_{\rm pole}$, and $\tau_{\rm pole}$ agree exactly between AMR and Cartesian runs. The $\sim$ 3–5% amplitude difference is entirely attributable to the octree voxelisation of the sphere boundary: leaf cells that straddle the sphere edge carry only the average density, softening the sharp boundary. No normalisation error is present.

11. Summary of Key Design Choices

1. **Flat linear octree.** All cells (internal and leaf) are stored in a single 1-indexed Fortran array. Parent–child relationships are maintained through integer index arrays. This avoids pointer indirection and is cache-friendly.

2. **Precomputed face-neighbour table.** Built once before the simulation loop. Boundary crossings cost O(1) (plus O($\Delta\ell$) descent into finer cells if the neighbour is more refined).

3. **Per-cell Voigt parameters.** Each leaf stores its own $\Delta\nu_D$ and a, supporting arbitrary temperature distributions without any approximation.

4. **Frequency shift at every cell crossing.** The comoving-frame frequency is updated at each boundary, so the Voigt opacity is always evaluated in the correct local frame, including bulk velocity effects.

5. **Single floating-point accumulation of displacement.** The photon position is updated as $\mathbf{r}_0 + d\,\hat{k}$ rather than as a sequence of increments, preserving numerical precision across many cell crossings.

6. **Procedure pointers for AMR/Cartesian switching.** The simulation loop and photon injection are unchanged. AMR physics is activated solely by reassigning Fortran procedure pointers in `setup_v2c.f90`.

7. **Pole-traversal tau normalisation.** The AMR raytrace kernel itself is used to compute $\tau_{\rm pole}$, giving an exact normalisation that is consistent with the Cartesian convention and avoids volume-average errors.